COMPANION DOCUMENT · ARCHITECTURE DEEP-DIVE

How MCP, RAG, and the LLM Connect in FinPulse

The first two documents explained each piece on its own. This one explains the wiring between them — how a click in the browser and a call from an AI agent both end up running through the exact same retrieval-then-generation pipeline.

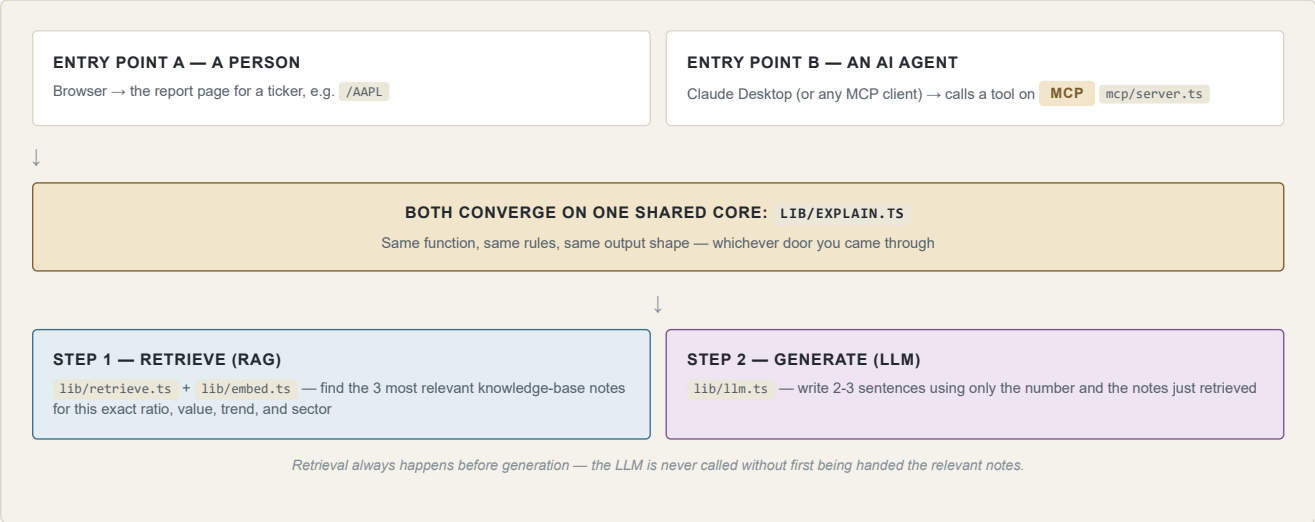
MCP RAG LLM

Companion to: FinPulse Full Documentation

Project location: F:\projects\finpulse

01 The One Diagram That Explains Everything

Two completely different "front doors" lead into FinPulse — a person using the website, and an AI agent using the MCP server. Both doors open onto the **same room**: one shared piece of logic that does retrieval (`RAG`) and then writing (`LLM`). Nothing about that shared logic changes depending on which door was used.



That's the whole idea of this document in one picture. Everything from here is unpacking each connection in that diagram in detail, plus one real, traced example.

02 The Shared Core — Why This Design Choice Matters

It would have been easy to write the explanation logic twice — once for the website, once for the MCP server. FinPulse deliberately doesn't do that. There is exactly **one** function that knows how to go from "a ratio and its value" to "a written explanation," and both entry points call it:

FUNCTION	FILE	CALLED BY THE WEBSITE?	CALLED BY MCP?
`explainCategory()`	`lib/explain.ts`	**Yes** — via `lib/cachedData.ts`	**Yes** — directly, no caching layer
`explainSingleRatio()`	`lib/explain.ts`	No (not needed — the website always has a full report)	**Yes** — the `explain_ratio` tool
`retrieveNotesBatch()`	`lib/retrieve.ts`	**Yes** (inside `explainCategory`)	**Yes** (inside both functions above)
`complete()`	`lib/llm.ts`	**Yes**	**Yes**

WHAT'S DIFFERENT BETWEEN THE TWO DOORS

Only the thin outer layer differs. The website wraps calls in a caching layer (`lib/cachedData.ts`) so repeated requests for the same numbers don't re-trigger the AI, and renders the result as an HTML page. The MCP server skips caching (each tool call is meant to be a fresh, direct answer to an agent) and returns the result as plain JSON or text instead of a web page. Neither of those differences touch the RAG or LLM logic itself.

03 Connection Point 1 — How RAG Feeds the LLM

This is the most important connection in the whole system: **RAG never talks to the user, and the LLM never talks to the knowledge base directly.** RAG's only job is to hand a small, relevant packet of text to the LLM before it writes anything.

What RAG produces

For one ratio — say, Apple's current ratio, currently 0.89, trending stable — `retrieveNotesBatch()` builds a short search phrase and finds the most relevant notes from the 42-note knowledge base:

```
Query built:  "current ratio 0.89 stable trend technology sector liquidity"

Notes found:
  [current-ratio]                    — what the ratio means, healthy ranges
  [current-ratio-sector-exceptions]  — why some sectors run below 1.0 on purpose
  [trend-liquidity]                  — what a stable trend implies here
```

How that becomes part of the LLM's instructions

Those retrieved notes are pasted directly into the text the LLM receives, under a clearly labeled "REFERENCE NOTES" section, alongside the already-computed number and trend. A simplified version of what the LLM actually receives looks like this:

```
COMPANY: Apple Inc. (Technology)

COMPUTED RATIOS:
- Current ratio: 0.89 (2025), trend: stable

REFERENCE NOTES:
[current-ratio] The current ratio measures whether a company can
cover its short-term obligations with short-term assets...

[current-ratio-sector-exceptions] Software and SaaS companies tend
to run high current ratios... (etc.)

Rules:
- Use only the numbers given above. Do not calculate anything.
- If the reference notes mention a sector exception that applies
  here, say so.
```

The LLM never sees the other 39 notes, and never sees Apple's raw financial statements. It only ever sees this one small, pre-filtered packet — which is exactly what keeps its explanations grounded instead of generic.

04 Connection Point 2 — How MCP Plugs Into RAG + LLM

The MCP server doesn't have its own separate idea of how to explain a ratio — it's a thin wrapper that hands off to the exact functions described in Section 02. The important design decision is that **each of the three MCP tools uses a different slice** of the RAG + LLM pipeline, on purpose:

MCP TOOL	RAG	LLM	WHY
`get_ratios`	not used	not used	Just runs the plain-code calculations. An agent chaining many calls usually wants fast, free numbers — not prose.
`explain_ratio`	**used**	**used**	One ratio, no company lookup — retrieves notes for that one ratio, then writes one explanation.
`analyze_ticker`	**used**	**used**	The full pipeline, run once per category (4 times), same as the website.

This is the same reasoning as an à la carte menu: an agent can order "just the numbers" (`get_ratios`), "one explained dish" (`explain_ratio`), or "the whole meal" (`analyze_ticker`) — and only the tools that actually need RAG and the LLM pay the cost (in time and API usage) of using them.

What connects an MCP tool call to the LLM, concretely

1. `mcp/server.ts`
   An agent calls the `explain_ratio` tool with `{ ratio: "current_ratio", value: 0.89, sector: "Technology" }`.
2. `mcp/server.ts → lib/explain.ts`
   The tool handler calls `explainSingleRatio()` directly — no HTTP request, no caching, just a normal function call inside the same running program.
3. `lib/explain.ts → lib/retrieve.ts`
   `explainSingleRatio()` calls `retrieveNotes()` to fetch the top 3 relevant knowledge-base notes for this ratio and sector.
4. `lib/explain.ts → lib/llm.ts`
   Those notes plus the value are handed to `complete()`, which calls Groq (or Cerebras on failure) and returns written text.
5. `lib/explain.ts → mcp/server.ts`
   The number guard checks the text, then the finished explanation is handed back to the agent as the tool's result.

05 Connection Point 3 — How the Website Plugs Into the Same Pipeline

The website's version of this connection looks almost identical — it just arrives through a different door, and adds one extra step (caching) that the MCP server skips.

1 `app/[ticker]/page.tsx`

A person opens `/AAPL`. The page immediately fetches and computes all the ratios (no AI yet) and renders them right away.

2 `app/components/CategoryExplanations.tsx`

For each of the 4 categories, a separate component starts asking for that category's explanations — while the numbers are already visible on screen.

3 `lib/cachedData.ts`

Before doing any real work, it checks: have we already generated an explanation for these *exact* ratio values before? If yes, the cached text is reused instantly and RAG/LLM are skipped entirely for this request.

4 `lib/explain.ts → lib/retrieve.ts`

If not cached: exactly the same `retrieveNotesBatch()` call as Section 04 — one embedding request covering every ratio in that category at once.

5 `lib/explain.ts → lib/llm.ts`

Exactly the same `complete()` call, same Groq-then-Cerebras logic, same number guard.

6 `app/components/CategoryExplanations.tsx`

The finished text streams into the page under each ratio — the page never had to sit blank waiting for it.

THE ONE REAL DIFFERENCE, RESTATED

Step 3 (the cache check) only exists on the website's path. That's a deliberate choice: a website gets repeat visits to the same ticker from many different people, so caching saves real time and money. An MCP tool call is treated as a one-off request from an agent that wants a fresh answer right now, so it skips straight to RAG and the LLM every time.

06 Traced End to End: One Real Example

To make the connection completely concrete, here is the full, real path for one specific thing: explaining Apple's current ratio, first through the website, then through MCP — placed side by side so the shared middle section is obvious.

STAGE	WEBSITE PATH	MCP PATH
Entry	Person visits `/AAPL`	Agent calls `explain_ratio`
Numbers ready	`lib/yahoo.ts` + `lib/ratios.ts` already ran for the whole report	The agent already supplies the value directly — no fetch needed
Cache check	**Yes** — `lib/cachedData.ts`	Skipped
RAG — find notes	**identical call** — `lib/retrieve.ts` + `lib/embed.ts`	
LLM — write text	**identical call** — `lib/llm.ts`, Groq → Cerebras fallback	
Trust check	**identical** — the number guard in `lib/explain.ts`	
Result shown as	Text under the ratio row on the page	Plain text returned to the agent

The middle three rows — retrieval, generation, and the trust check — are not just "similar," they are the exact same lines of code running. That's the entire point of building it this way: whichever door the request came through, RAG and the LLM behave identically, and the explanation you'd get from asking Claude Desktop is the same explanation you'd see on the website for the same numbers.

07 One-Page Recap

THE SHORT VERSION

- `MCP` is the **door** — a standard way for an AI agent to reach FinPulse's logic, alongside the website's own door.
- `RAG` is the **librarian** — it runs first, and hands the LLM a small, relevant packet of trusted notes instead of letting it rely on general knowledge.
- `LLM` is the **writer** — it runs last, and only ever writes from the number and notes it was just handed.
- Both doors — the website and MCP — lead into **one shared pipeline** (`lib/explain.ts`), so the RAG and LLM behavior never diverges depending on how the request arrived.
- The only thing that differs between the two doors is caching (website: yes, for repeat visitors) and output format (website: a rendered page; MCP: plain text/JSON for an agent to use).